

Brain Tumor AI Pipeline Integration Guide

How to Combine All Models Into a Working Pipeline

Backend Development Reference

Graduation Project — April 2026

Table of Contents

1. Overview — What the Pipeline Does
2. Architecture — How the Three Models Connect
3. Required Files and Directory Structure
4. Loading All Models
5. Stage 1: Classification — Input, Preprocessing, Output
6. Stage 2: Segmentation — Input, Preprocessing, Output
7. Stage 3: Size and Location Extraction
8. Stage 4: Treatment Suggestion — Input, Preprocessing, Output
9. Full Pipeline Flow — Step by Step
10. Error Handling and Edge Cases
11. API Contract — Input/Output Specification for Backend
12. Deployment Notes

1. Overview — What the Pipeline Does

The Brain Tumor AI Pipeline is an end-to-end system that takes two inputs — a patient intake form and a brain MRI image — and produces three clinical outputs: a treatment plan recommendation, an urgency level, and a triage tier. The system is designed so that no medical professional is needed to operate it (the patient provides all inputs), but all outputs are explicitly labeled as suggestions that require professional review.

The pipeline consists of three deep learning / machine learning models that execute sequentially. Each model's output becomes input for the next stage. If any stage fails or produces a "no tumor" result, the pipeline stops early and reports what it found. This document explains exactly how each model works, what it expects as input, what it produces as output, and how to connect them in a backend application.

2. Architecture — How the Three Models Connect

2.1 Pipeline Stages

Stage	Model	Framework	Input	Output
1. Classification	EfficientNetB0	TensorFlow/Keras	MRI image (224x224 RGB)	tumor_type + confidence
2. Segmentation	U-Net	TensorFlow/Keras	MRI image (256x256 gray)	Binary mask (256x256)
3a. Size extraction	OpenCV (no ML)	OpenCV	Binary mask	tumor_size_mm2, diameters
3b. Location extraction	Rule-based (no ML)	OpenCV	Binary mask + tumor_type	tumor_location, hemisphere
4. Treatment suggestion	XGBoost (x3)	scikit-learn/xgboost	17 features (form + MRI)	treatment, urgency, triage

2.2 Data Flow

The data flows through the pipeline as follows. The patient fills a form providing 13 features (age, sex, smoking, comorbidities, symptoms, functional status). Separately, the patient uploads an MRI image. The image passes through the classification model, which outputs a tumor type and confidence score. If the result is "notumor" or confidence is below 80%, the pipeline can stop or warn. Otherwise, the same image passes through the segmentation model, which produces a binary mask. Two extraction functions process the mask to compute tumor_size_mm2, tumor_location, and hemisphere. These 4 MRI-extracted features are combined with the 13 form features to create a 17-feature vector. This vector is passed to 3 XGBoost models, which output treatment_plan, urgency_level, and triage_tier.

Critical point for backend developers: The classification and segmentation models process the **same image** but with **different preprocessing**. Classification needs RGB 224x224 with EfficientNet preprocessing. Segmentation needs grayscale 256x256 normalized to [0,1]. The image must be preprocessed twice, differently, for each model.

3. Required Files and Directory Structure

3.1 Model Files

File	Stage	Size	Format
brain_tumor_classifier.h5	Classification	~16 MB	Keras HDF5
calibration.json	Classification	< 1 KB	JSON
tumor_unet_best_dice.h5	Segmentation	~120 MB	Keras HDF5
treatment_plan_xgb_model.pkl	Suggestion	6.9 MB	joblib pickle
urgency_level_xgb_model.pkl	Suggestion	809 KB	joblib pickle
triage_tier_xgb_model.pkl	Suggestion	1.6 MB	joblib pickle
scaler.pkl	Suggestion	1 KB	joblib pickle
model_metadata.json	Suggestion	2 KB	JSON

3.2 Recommended Directory Layout

```
models/  
  classification/  
    brain_tumor_classifier.h5  
    calibration.json  
  segmentation/  
    tumor_unet_best_dice.h5  
  suggestion/  
    treatment_plan_xgb_model.pkl  
    urgency_level_xgb_model.pkl  
    triage_tier_xgb_model.pkl  
    scaler.pkl  
    model_metadata.json
```

3.3 Python Dependencies

Package	Version	Used By
tensorflow	>= 2.15	Classification + Segmentation models
opencv-python (cv2)	>= 4.8	Image preprocessing, contour extraction
numpy	>= 1.24	Array operations throughout
pandas	>= 2.0	Feature DataFrame for suggestion model
scikit-learn	>= 1.3	LabelEncoder, StandardScaler
xgboost	>= 2.0	Treatment suggestion models
joblib	>= 1.3	Loading .pkl model files

4. Loading All Models

All models should be loaded once at application startup, not on every request. Loading takes 5-10 seconds total but should only happen once. Here is the complete loading sequence:

4.1 Classification Model

```
from tensorflow.keras.models import load_model
import json

classifier = load_model('models/classification/brain_tumor_classifier.h5')
with open('models/classification/calibration.json') as f:
    calib = json.load(f)
TEMPERATURE = calib['temperature'] # 1.8874
CLASS_NAMES = calib['classes']     # ['glioma', 'meningioma', 'notumor', 'pituitary']
```

4.2 Segmentation Model

```
segmentor = load_model('models/segmentation/tumor_unet_best_dice.h5', compile=False)
```

Note: compile=False is required because the model was saved with a custom Dice metric that is not available at load time. The model is used for inference only (predict), so compilation is not needed.

4.3 Suggestion Models

```
import joblib
import numpy as np
from sklearn.preprocessing import LabelEncoder

suggestion_models = {
    'treatment_plan': joblib.load('models/suggestion/treatment_plan_xgb_model.pkl'),
    'urgency_level': joblib.load('models/suggestion/urgency_level_xgb_model.pkl'),
    'triage_tier': joblib.load('models/suggestion/triage_tier_xgb_model.pkl'),
}
scaler = joblib.load('models/suggestion/scaler.pkl')
with open('models/suggestion/model_metadata.json') as f:
    meta = json.load(f)

# Rebuild encoders from metadata
feature_encoders = {}
for col, classes in meta['feature_encoders'].items():
    le = LabelEncoder()
    le.classes_ = np.array(classes)
    feature_encoders[col] = le

target_encoders = {}
for col, classes in meta['target_encoders'].items():
    le = LabelEncoder()
    le.classes_ = np.array(classes)
    target_encoders[col] = le
```

5. Stage 1: Classification

5.1 Input Requirements

Property	Requirement
Image format	Any format OpenCV can read (.jpg, .png, .bmp)
Color space	RGB (converted from BGR after cv2.imread)

Resize to	224 x 224 pixels
Normalization	EfficientNet preprocess_input (NOT /255.0)
Batch shape	(1, 224, 224, 3) — single image batch

5.2 Preprocessing Code

```
import cv2
import numpy as np
from tensorflow.keras.applications.efficientnet import preprocess_input

img = cv2.imread(image_path)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img_resized = cv2.resize(img_rgb, (224, 224))
img_input = preprocess_input(img_resized.astype(np.float32))
img_batch = np.expand_dims(img_input, axis=0)
```

5.3 Prediction and Temperature Calibration

The classifier outputs raw softmax probabilities. These must be calibrated using the saved temperature parameter to produce reliable confidence scores. Without calibration, the model may output 99% confidence on wrong predictions.

```
probs = classifier.predict(img_batch, verbose=0)[0]

# Temperature scaling calibration
logits = np.log(probs + 1e-12)
scaled_logits = logits / TEMPERATURE # TEMPERATURE = 1.8874
exp_logits = np.exp(scaled_logits - np.max(scaled_logits))
calibrated_probs = exp_logits / exp_logits.sum()

pred_idx = np.argmax(calibrated_probs)
tumor_type = CLASS_NAMES[pred_idx]
confidence = float(calibrated_probs[pred_idx])
conf_level = 'HIGH' if confidence >= 0.80 else 'LOW'
```

5.4 Output

Field	Type	Values	Example
tumor_type	string	glioma, meningioma, notumor, pituitary	glioma
confidence	float	0.0 to 1.0	0.66
conf_level	string	HIGH or LOW	LOW

5.5 Decision Point

If tumor_type is "notumor", the pipeline stops. Report to user: no tumor detected, no further analysis needed. If conf_level is "LOW", the pipeline continues but the result should display a warning to the user. The 80% threshold was chosen because the classification model achieves 100% accuracy on high-confidence predictions in testing.

6. Stage 2: Segmentation

6.1 Input Requirements

Property	Requirement
Image format	Same image file used for classification
Color space	Grayscale (single channel)
Resize to	256 x 256 pixels
Normalization	Divide by 255.0 (scale to [0, 1])
Batch shape	(1, 256, 256, 1) — single grayscale image batch

6.2 Preprocessing Code

```
img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
img_resized = cv2.resize(img, (256, 256))
img_input = img_resized.astype(np.float32) / 255.0
img_batch = img_input.reshape(1, 256, 256, 1)
```

Critical difference from classification: Classification uses RGB with EfficientNet preprocessing. Segmentation uses grayscale with simple /255.0 normalization. Using the wrong preprocessing will produce garbage output from either model.

6.3 Prediction and Binarization

```
pred_mask = segmentor.predict(img_batch, verbose=0)[0, :, :, 0]
binary_mask = (pred_mask > 0.5).astype(np.uint8)
```

The segmentation model outputs a probability map (values 0.0 to 1.0) where each pixel represents the probability of being tumor tissue. Thresholding at 0.5 converts this to a binary mask: 1 = tumor, 0 = healthy.

6.4 Output

Field	Type	Shape	Values
binary_mask	numpy array (uint8)	256 x 256	0 (healthy) or 1 (tumor)
img_resized	numpy array (uint8)	256 x 256	Original grayscale for overlay

7. Stage 3: Size and Location Extraction

These are not ML models — they are deterministic OpenCV functions that extract measurements from the binary mask. They do not require GPU and execute in under 1 millisecond.

7.1 Tumor Size Extraction

The `calculate_tumor_size` function takes the binary mask and a `pixel_spacing` parameter (default 0.5 mm/pixel) and performs four steps:

- **Step 1 — Pixel count:** Count all non-zero pixels in the mask. This is the raw area in pixels.
- **Step 2 — Physical area:** Multiply pixel count by `pixel_spacing` squared. With 0.5 mm/pixel, 1 pixel = 0.25 mm².
- **Step 3 — Contour area:** Find the largest contour using `cv2.findContours` and compute its geometric area with `cv2.contourArea`. More accurate than pixel counting.
- **Step 4 — Ellipse fitting:** If the contour has ≥ 5 points, fit an ellipse with `cv2.fitEllipse` to estimate major and minor diameters in mm.

Output:

Field	Type	Unit	Range
area_mm2	float	mm2	15 - 4,000 (clamp to this range)
major_diameter	float	mm	0 - ~100
minor_diameter	float	mm	0 - ~100

Important: The suggestion model expects `tumor_size_mm2` in the range 15 to 4,000. If the extraction returns a value outside this range, clamp it: values below 15 become 15, values above 4,000 become 4,000.

7.2 Tumor Location Extraction

The `get_tumor_location` function determines the anatomical brain region using the mask centroid position.

Special case — Pituitary: If `tumor_type` is "pituitary", always return `location="Sellar"` and `hemisphere="M"` regardless of mask position. This is a medical fact, not a computational decision.

For glioma and meningioma: The function finds the largest contour, computes its centroid using image moments (`cv2.moments`), normalizes the centroid to [0,1] relative to image dimensions, and maps to a region:

Normalized Y	Condition	Region
< 0.35	Any x	Frontal
0.35 - 0.55	$x < 0.25$ or $x > 0.75$	Temporal
0.35 - 0.55	Center	Frontal
0.55 - 0.75	Any x	Parietal
≥ 0.75	Any x	Occipital

Hemisphere is determined by normalized X:

- $x < 0.42 \rightarrow$ Left (L)

- $0.42 \leq x \leq 0.58 \rightarrow \text{Midline (M)}$
- $x > 0.58 \rightarrow \text{Right (R)}$

Output:

Field	Type	Values
location	string or None	Frontal, Temporal, Parietal, Occipital, Sellar
hemisphere	string or None	L, R, M
detail	string	e.g. 'Right Frontal Lobe'

Fallback: If no contour is found (mask is all zeros) or location is None, the backend should use defaults: location="Frontal", hemisphere="M". These defaults are safe because they do not trigger any special treatment rules.

8. Stage 4: Treatment Suggestion

8.1 Input: Assembling the 17-Feature Vector

The suggestion model requires exactly 17 features in a specific order. Thirteen come from the patient form and 4 come from the MRI analysis stages. The backend must combine them into a single dictionary:

```
patient_data = {
    # ■■ From patient form (13 features) ■■
    'age': 55,                # int, 18-89
    'sex': 'M',               # str, 'M' or 'F'
    'smoking_status': 'Former', # str, 'Never'/'Former'/'Current'
    'diabetes': 0,            # int, 0 or 1
    'hypertension': 1,        # int, 0 or 1
    'prior_cancer': 0,        # int, 0 or 1
    'prior_brain_surgery': 0, # int, 0 or 1
    'immunosuppressed': 0,    # int, 0 or 1
    'seizures': 1,            # int, 0 or 1
    'headache_severity': 7,   # int, 0-10
    'symptom_duration_weeks': 4, # int, 1-52
    'functional_status': 'Independent', # str
    'neurological_symptoms': 0, # int, 0 or 1

    # ■■ From MRI analysis (4 features) ■■
    'tumor_type': 'glioma',    # from Stage 1
    'tumor_location': 'Frontal', # from Stage 3b
    'tumor_size_mm2': 450.0,   # from Stage 3a
    'hemisphere': 'L',         # from Stage 3b
}
```

8.2 Preprocessing Steps (Inside predict_action)

The predict_action function performs three preprocessing steps before prediction:

Step 1 — Encode categorical features: Six string features (sex, smoking_status, functional_status, tumor_type, tumor_location, hemisphere) are converted to integers using the saved LabelEncoders. For example, 'glioma' becomes 0, 'meningioma' becomes 1, 'pituitary'

becomes 2. The exact mapping is stored in model_metadata.json under 'feature_encoders'.

Step 2 — Enforce column order: The DataFrame columns must be in the exact order used during training. The correct order is stored in model_metadata.json under 'feature_columns'. Incorrect ordering will silently produce wrong predictions.

Step 3 — Scale numeric features: Eleven numeric features are standardized using the saved StandardScaler. Only the features listed in meta['numeric_columns'] are scaled. Categorical features (already encoded as integers) are NOT scaled.

8.3 Prediction

Three separate XGBoost models run on the same preprocessed input:

```
results = {}
for target in ['treatment_plan', 'urgency_level', 'triage_tier']:
    pred = suggestion_models[target].predict(row_scaled)[0]
    label = target_encoders[target].inverse_transform([pred])[0]
    results[target] = label
```

8.4 Output

Field	Type	Possible Values
treatment_plan	string	Surgery, Watch & wait, Palliative, SRS, Dopamine agonist, Surgery+RT, Trans-sphenoidal, Biopsy
urgency_level	string	Elective, Urgent, Emergency
triage_tier	string	Routine, Urgent, Emergency

9. Full Pipeline Flow — Step by Step

This section describes the exact sequence a backend endpoint should follow:

Step 1: Receive patient form data (13 fields) and MRI image file from frontend.

Step 2: Validate form inputs — check types, ranges, and valid values (see Section 11).

Step 3: Save uploaded image temporarily to disk (required by cv2.imread).

Step 4: Run classification: preprocess image as RGB 224x224, predict, apply temperature calibration.

Step 5: Check result. If tumor_type == 'notumor': return {tumor_type: 'notumor', confidence, message: 'No tumor detected'}. Pipeline stops.

Step 6: Run segmentation: preprocess same image as grayscale 256x256, predict, binarize at 0.5.

Step 7: Extract size: pass binary mask to calculate_tumor_size(). Clamp area_mm2 to [15, 4000].

Step 8: Extract location: pass binary mask and tumor_type to get_tumor_location(). Handle None with defaults.

Step 9: Assemble 17-feature dictionary: merge 13 form fields + tumor_type + tumor_location + tumor_size_mm2 + hemisphere.

Step 10: Run suggestion: pass dictionary to predict_action(). Get treatment_plan, urgency_level, triage_tier.

Step 11: Generate overlay image (optional for frontend display): resize original to 256x256, color mask pixels red, blend at 70/30.

Step 12: Return JSON response with all results to frontend.

10. Error Handling and Edge Cases

Scenario	How to Handle
Image cannot be read by cv2	Return error: 'Invalid image format'
Classification returns notumor	Stop pipeline, return notumor result only
Classification confidence < 80%	Continue but add warning flag to response
Segmentation mask is all zeros	Use defaults: size=100, location=Frontal, hemisphere=M
tumor_size_mm2 < 15 or > 4000	Clamp to range boundaries
Location extraction returns None	Use default: location=Frontal, hemisphere=M
Categorical value not in encoder	Return error: 'Invalid value for [field]'
Missing form field	Return error: 'Missing required field: [field]'
Pituitary tumor type detected	Force location=Sellar, hemisphere=M (medical rule)

11. API Contract — Input/Output Specification for Backend

11.1 Request Input

Form fields (13):

Field	Type	Valid Values
age	integer	18 to 89
sex	string	'M' or 'F'
smoking_status	string	'Never', 'Former', 'Current'
diabetes	integer	0 or 1
hypertension	integer	0 or 1
prior_cancer	integer	0 or 1
prior_brain_surgery	integer	0 or 1
immunosuppressed	integer	0 or 1

seizures	integer	0 or 1
headache_severity	integer	0 to 10
symptom_duration_weeks	integer	1 to 52
functional_status	string	'Independent', 'Some help', 'Significant help', 'Bed-bound'
neurological_symptoms	integer	0 or 1

Image file: 1 brain MRI image (.jpg or .png), any resolution (will be resized internally).

11.2 Response Output (JSON)

```
{
  'tumor_type': 'glioma',
  'confidence': 0.66,
  'confidence_level': 'LOW',
  'tumor_size_mm2': 99.8,
  'tumor_location': 'Frontal',
  'hemisphere': 'R',
  'location_detail': 'Right Frontal Lobe',
  'major_diameter_mm': 18.0,
  'minor_diameter_mm': 8.4,
  'treatment_plan': 'Watch & wait',
  'urgency_level': 'Urgent',
  'triage_tier': 'Urgent',
  'overlay_image': '<base64 encoded PNG>' // optional
}
```

12. Deployment Notes

12.1 Memory Requirements

All models loaded simultaneously require approximately 200 MB of RAM. The segmentation model is the largest component (~120 MB). For a web server, this is a one-time cost at startup. GPU is not required for inference — all models can run on CPU with acceptable latency (under 2 seconds per request).

12.2 Inference Latency

Stage	CPU Time	GPU Time
Classification (EfficientNetB0)	~500 ms	~50 ms
Segmentation (U-Net)	~800 ms	~100 ms
Size + Location extraction	< 1 ms	< 1 ms
Treatment suggestion (3x XGBoost)	< 10 ms	N/A (CPU only)
Total pipeline	~1.3 seconds	~200 ms

12.3 Thread Safety

TensorFlow models (classifier, segmentor) are NOT thread-safe. If the backend uses multiple threads, wrap predictions in a lock or use TensorFlow's thread-safe serving mode. XGBoost models are thread-safe for prediction. The scaler and encoders are stateless after loading and are safe for concurrent use.

12.4 Model Versioning

All model files should be stored together as a versioned set. If any model is retrained, all dependent models should be tested together. In particular, if the segmentation model changes, the `tumor_size_mm2` distribution may shift, which could affect suggestion model accuracy. The `model_metadata.json` file acts as a manifest — its content must match the models it accompanies.